Практическая работа №6. Асинхронная работа с SQL Server

Цель работы

Научиться выполнять асинхронные операции с SQL Server из C# с помощью `async/await`, `OpenAsync()`, `ExecuteScalarAsync()`, `ExecuteReaderAsync()`, `ReadAsync()` и `ExecuteNonQueryAsync()`, а также обрабатывать ошибки и отмену операций.

База данных: `coffee_shop`. **Ориентировочное время:** 80–90 минут.

Задание

Часть 1. Подготовка проекта

Создайте консольное приложение на C# или используйте проект предыдущих работ. Установите `Microsoft.Data.SqlClient`.

Убедитесь, что в проекте доступен асинхронный `Main` (современный .NET поддерживает `await` в инструкциях верхнего уровня).

```
using Microsoft.Data.SqlClient;
string connectionString =
    "Server=localhost;Database=coffee_shop;Trusted_Connection
=True;TrustServerCertificate=True;";
```

При необходимости замените параметры подключения на свои.

Часть 2. Асинхронное подключение и количество товаров

Создайте метод, который открывает соединение асинхронно, выполняет `SELECT COUNT(\*) FROM products` и возвращает число товаров.

```
static async Task<int> GetProductCountAsync(string connection
String)
{
    await using SqlConnection connection =
        new SqlConnection(connectionString);
    await connection.OpenAsync();
    using SqlCommand command = new SqlCommand(
        "SELECT COUNT(*) FROM products", connection);
    return (int)(await command.ExecuteScalarAsync())!;
}
```

Вызовите метод:

```
int count = await GetProductCountAsync(connectionString);
Console.WriteLine($"Количество товаров: {count}");
```

Объясните, зачем здесь нужны `Task\<int\>` и `await`.

Часть 3. Асинхронное чтение каталога

Создайте метод `ShowProductsAsync(string connectionString)`, который получает товары из `products` и выводит их в консоль.

Используйте запрос:

```
SELECT id, name, price, is_available
FROM products
ORDER BY name;
```

Пример чтения результата:

```
await using SqlDataReader reader = await command.ExecuteReade
rAsync();
while (await reader.ReadAsync())
```

```
{
    int id = reader.GetInt32(0);
    string name = reader.GetString(1);
    decimal price = reader.GetDecimal(2);
    bool available = reader.GetBoolean(3);
    Console.WriteLine($"{id}. {name} — {price} руб. | {(avail
able ? "Есть" : "Нет")}");
}
```

Перед созданием `reader` самостоятельно откройте соединение через `OpenAsync()` и создайте `SqlCommand`. Если строк нет, выведите сообщение `Товары не найдены.`.

Часть 4. Асинхронный поиск по минимальной цене

Реализуйте метод `ShowProductsFromPriceAsync(string connectionString, decimal minPrice)`.

Используйте параметризованный запрос:

```sql
SELECT id, name, price
FROM products
WHERE price >= @min_price
ORDER BY price;
```

Требования:

1. Пользователь вводит минимальную цену.
2. Значение передаётся через параметр `@min_price`.
3. Запрос выполняется через `ExecuteReaderAsync()`.
4. Строки читаются через `ReadAsync()`.
5. При отсутствии совпадений выводится понятное сообщение.

Часть 5. Асинхронное обновление товара

Создайте метод `UpdateProductAvailabilityAsync(string connectionString, int productId, bool isAvailable)`.

Используйте запрос:

```sql
UPDATE products
SET is_available = @is_available
WHERE id = @id;
```

Создайте `SqlCommand` с параметрами и вызовите `ExecuteNonQueryAsync()`. Проверьте количество изменённых строк. Если оно равно нулю, сообщите, что товар не найден.

Проверьте работу метода, изменив доступность одного товара. Затем верните прежнее значение.

Часть 6. Обработка ошибок

Добавьте `try/catch` вокруг вызовов методов работы с базой.

Для ошибок SQL Server используйте `SqlException`, для некорректного ввода пользователя — безопасную проверку `int.TryParse()` и `decimal.TryParse()`.

Не формируйте SQL-запросы через конкатенацию с пользовательским вводом.

Часть 7. Самостоятельная часть

Создайте консольное меню **«Асинхронный каталог кофейни»** со следующими операциями:

1. Показать все товары с ценами и доступностью.
2. Показать только доступные товары.
3. Найти товары по части названия (параметр `@search` и оператор `LIKE`).
4. Показать число клиентов (таблица `customers`).
5. Изменить доступность товара по идентификатору.
6. Завершить программу.

Условия:

- все обращения к SQL Server выполняются через асинхронные методы ADO.NET;
- для ввода значений используются параметры;

- запросы на чтение нескольких записей используют `ExecuteReaderAsync()` и `ReadAsync()`;
- запрос количества клиентов использует `ExecuteScalarAsync()`;
- изменение доступности использует `ExecuteNonQueryAsync()`;
- программа сообщает об отсутствии результатов и об ошибках ввода;
- соединения и reader корректно освобождаются.

Часть 8. Отмена длительной операции

Дополните **один** асинхронный метод необязательным аргументом `CancellationToken` и передайте его в `OpenAsync(token)` и `ExecuteReaderAsync(token)` или `ExecuteScalarAsync(token)`.

При вызове создайте `CancellationTokenSource` и установите `CancelAfter(TimeSpan.FromSeconds(5))`. Обработайте `OperationCanceledException`.

Если запрос завершается быстро и отмена не успевает сработать — это нормально. Не пытайтесь искусственно замедлять рабочий SQL Server.

Что необходимо продемонстрировать преподавателю

- Использование `async`, `await`, `Task` и асинхронных методов ADO.NET;
- асинхронное открытие соединения и чтение строк;
- параметризованный поиск;
- асинхронное выполнение `UPDATE`;
- работу меню и проверку пользовательского ввода;
- корректную обработку ошибок и освобождение ресурсов.

Выводы

Письменно ответьте:

1. В чём преимущество асинхронного ожидания ответа SQL Server?
2. Чем `ExecuteReaderAsync()` отличается от `ReadAsync()`?

3. Почему метод возвращает `Task\<int\>`, а после `await` получается `int`?
4. Почему параметры нужны и в асинхронных запросах?
5. Для чего нужен `CancellationToken`?

Дополнительное задание

Создайте метод `GetCustomerOrdersAsync(string connectionString, int customerId)`, который с помощью `JOIN` выводит список заказов выбранного клиента (номер, дата, статус). Примените асинхронное чтение и параметр `@customer_id`.